

MODUL PRAKTIKUM PERTEMUAN 4

Implementasi Implementasi Validation dan Error Handling pada REST API

Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa mampu:

1. Memahami pentingnya validasi data pada sistem client–server.
2. Mengimplementasikan validation rules pada Laravel.
3. Mencegah data tidak valid masuk ke database.
4. Menghasilkan response error yang informatif.
5. Menguji validasi API menggunakan Postman.

Dasar Teori Singkat

1. Validasi Data pada Sistem Client–Server

Pada arsitektur client–server, server bertanggung jawab memastikan bahwa data yang dikirim client memenuhi aturan sistem.

Tanpa validasi dapat terjadi:

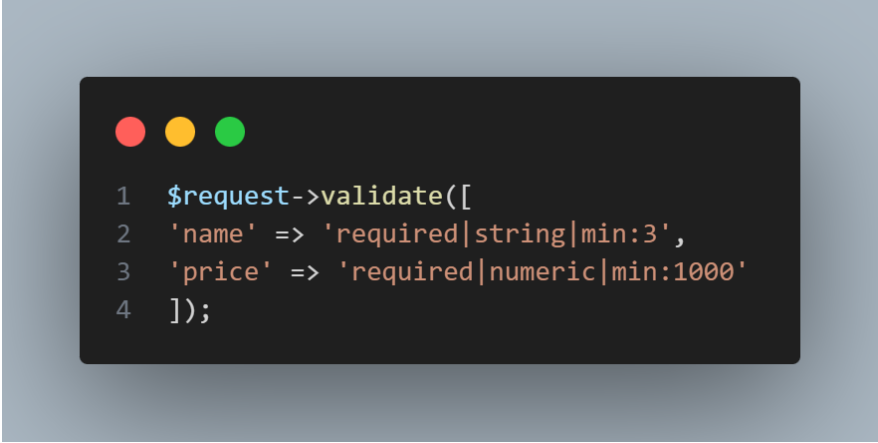
- data kosong tersimpan di database
- format data tidak sesuai
- error pada sistem
- ketidakkonsistenan data

Oleh karena itu server harus melakukan validasi sebelum menyimpan data.

2. Validation pada Laravel

Laravel menyediakan fitur validasi yang dapat digunakan untuk memeriksa data dari request client.

Contoh validasi pada Laravel:



```
1 $request->validate([
2     'name' => 'required|string|min:3',
3     'price' => 'required|numeric|min:1000'
4 ]);
```

3. Validation Rules

Beberapa aturan validasi yang umum digunakan adalah sebagai berikut.

Rule	Fungsi
required	field wajib diisi
string	memastikan data berupa teks
numeric	memastikan data berupa angka
min	nilai minimal
max	nilai maksimal
unique	memastikan data tidak duplikat

Alat dan Bahan

1. Laptop / komputer
2. Visual Studio Code
3. Laravel Project dari pertemuan sebelumnya
4. Database MySQL
5. Postman

Langkah Kerja Praktikum

A. Membuka ProductController

Buka file berikut:

app/Http/Controllers/Api/ProductController.php

Controller ini sudah dibuat pada pertemuan sebelumnya.

B. Menambahkan Validation pada Method Store

Ubah method store() menjadi seperti berikut.



```
1 public function store(Request $request)
2 {
3     $validated = $request->validate([
4         'name' => 'required|string|min:3|max:100',
5         'price' => 'required|numeric|min:1000|max:100000000',
6         'description' => 'nullable|string|max:255'
7     ]);
8
9     $product = Product::create($validated);
10
11     return response()->json([
12         'success' => true,
13         'message' => 'Product berhasil ditambahkan',
14         'data' => $product
15     ], 201);
16 }
```

Penjelasan:

- **required** memastikan field tidak boleh kosong
- **string** memastikan data berupa teks
- **numeric** memastikan data berupa angka
- **min dan max** membatasi nilai data

c. Menambahkan Validation pada Method Update

1. Ubah method **update()** sebagai berikut.

```
1 public function update(Request $request, $id)
2 {
3     $product = Product::findOrFail($id);
4
5     $validated = $request->validate([
6         'name' => 'sometimes|string|min:3|max:100',
7         'price' => 'sometimes|numeric|min:1000|max:100000000',
8         'description' => 'nullable|string|max:255'
9     ]);
10
11     $product->update($validated);
12
13     return response()->json([
14         'success' => true,
15         'message' => 'Product berhasil diperbarui',
16         'data' => $product
17     ]);
18 }
```

Pada proses update digunakan kata **sometimes** karena tidak semua field harus dikirim oleh client.

D. Menguji Validation Menggunakan Postman

1. Buka aplikasi **Postman**. Pilih method POST

<http://127.0.0.1:8000/api/products>

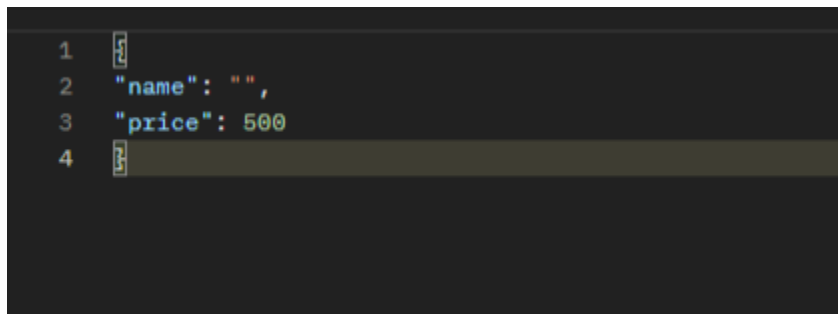
Body → raw → JSON



Contoh data yang benar

E. Pengujian Validation Error:

1. Coba kirimkan data yang salah :



2. Laporkan Hasl yang muncul jika pengujian berhasil/gagal.

E. Latihan

Kerjakan tugas berikut.

1. Tambahkan aturan validasi berikut pada field **name**

```
'name' => 'required|unique:products,name'
```

2. Coba tambahkan produk dengan nama yang sama dua kali.
3. Amati respon error yang diberikan oleh server.

F. Diskusi

Jawab pertanyaan berikut.

1. Mengapa validasi harus dilakukan pada sisi server?
2. Apa perbedaan validasi pada client dan server?
3. Mengapa sistem harus menolak data yang tidak valid?